

CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference

Shubhankar Vishnu Patil

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
shubhankarp258@gmail.com

Yash Suresh Kodgirwar

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
yashkodgirwar@gmail.com

Kunal Kundu

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India

Vivek Dnyaneshwar Joshi

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
vivekdjoshi05@gmail.com

Nirmalya Mukhopadhyay

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India

Abstract—Modern software development increasingly relies on Continuous Integration/Continuous Deployment (CI/CD) pipelines to deliver software rapidly, but automated security review at the pull request stage remains limited. Existing security tools rely on rigid static analysis or homogeneous multi-agent approaches, which may generate false positives or overlook complex vulnerabilities. This project will present CodeSheriff, an agentic security auditor that will analyze GitHub pull requests using structural, semantic, contextual, and SFI-based confidential execution analysis. A Bayesian inference engine will fuse evidence from multiple agents to generate confidence-aware vulnerability assessments, while an automated patch generation and verification module will recommend and validate fixes through GitHub and a web dashboard. The system will enhance CI/CD security by delivering reliable, explainable, and statistically grounded vulnerability detection while reducing manual review effort.

Index Terms—vulnerability detection, multi-agent systems, software security, continuous integration and continuous deployment

I. INTRODUCTION

In today's software industry, developers frequently push code to the main branch through Pull Requests (PRs). However, security analysis of each Pull Request is still largely performed manually, making the review process more time-consuming. Teams either use basic approaches that only catch known, simple mistakes, or they don't have enough time to properly review every pull request. This is a big problem because even one missed vulnerability can put the whole application at risk once the code is merged and deployed. On top of this, attackers are now also targeting the Continuous

Integration/Continuous Deployment (CI/CD) that's builds and runs the code.

Solutions that exist today don't fully solve this problem. Traditional static analysis approaches identify fixed patterns but are unable to catch tricky ones, so they generate a lot of false alarms. Some recent tools use multiple artificial intelligence (AI) agents, but all these agents basically read the same code in the same way, so they don't really give different opinions. When these approaches combine results, they just use simple voting, which doesn't tell you how confident the final answer actually is.

In this paper, we are proposing CodeSheriff to fix this problem. It uses four different types of agents that look at the code from different perspectives. Static agent checks the code structure, the meaning of the code is understood by the semantic agent, the context agent checks whether accepting this PR could bring back any vulnerability from previously accepted PR, and the runtime agent actually watches the code run inside a safe, isolated environment using software fault isolation (SFI). All this information is then combined using a Bayesian model, which gives a clear probability of whether the code is vulnerable, along with how confident the system is. This same safe environment also protects the CI/CD workflow, since it only releases secret keys when it is sure the correct, verified code is running.

The rest of this paper is structured as follows. Section II talks about related work already done in this area. Section III explains our proposed method and how the system is designed. Section IV describes the implementation and the labelled dataset. Section V presents and discusses the results,

and Section VI concludes the paper with directions for future work.

II. LITERATURE SURVEY

Early learning-based studies explored structural and sequential representations for vulnerability detection. Li and Yang [1] compared sequence and graph-based deep learning methods, identifying challenges in capturing long-range dependencies and generalizing to noisy data. CODE-SMASH [2] and XTransformer [3] employed hierarchical neural and transformer architectures to capture multi-level code representations and improve similarity-based detection, while remaining sensitive to computational cost and dataset characteristics. Abbasi and Shameli-Sendi [4] used Code Property Graph metrics with machine learning for interpretable vulnerability and vulnerability-type prediction, but their approach was limited to C/C++. DeepSecure [5] combined CodeBERT with bidirectional long short-term memory (BiLSTM) and convolutional neural network (CNN) layers for python vulnerability detection, incorporating robustness and confidence calibration, but remained language-specific and affected by noisy labels.

Transformer and large language model (LLM) based approaches have subsequently improved semantic analysis of source code. Shestov *et al.* [6] fine-tuned WizardCoder using parameter-efficient learning for vulnerability classification, but remained limited to function-level analysis. Tamberg and Bahsi and Gnieciak and Szandala [7], [8] benchmarked LLMs against vulnerability detection tasks and traditional static analysis tools, showing strong potential but persistent false positives, hallucinations, localization errors, and inference costs. Gülmez [9] surveyed code-generation LLMs and highlighted challenges in structural correctness, repository-level reasoning, and security. Chen *et al.* [10] and Su *et al.* [11] similarly identified limitations involving context windows, hallucination, dataset quality, generalization, and interpretability. Germano *et al.* [12] reviewed LLM-based vulnerability detection, repair, and explanation, noting that research remains more focused on detection than automated repair and explanation.

To address the limitations of isolated code analysis, recent studies have incorporated contextual and repository-level information. Antal *et al.* [13] evaluated retrieval augmented generation (RAG) for real-world Java vulnerability detection using retrieved vulnerable and secure examples, but observed false positives and retrieval sparsity. AegisGuard [14] incorporated contextual information for semantic vulnerability detection and risk stratification, while SCOUT [15] enabled interactive repository exploration to retrieve relevant code context. Göçmen *et al.* [16] integrated repository history and dependency analysis into pull-request reviews for change-impact estimation, although static dependency analysis cannot fully capture dynamic behavior.

Multi-agent approaches have further introduced collaborative security reasoning. Wei *et al.* [17] used specialized roles and collaborative reasoning for vulnerability auditing, while MAVUL [18] combined vulnerability analysis, security architecture review, and iterative evaluation. Reddy *et al.* [19] ex-

plored knowledge-based agentic AI for automated vulnerability triage, and Syed *et al.* [20] investigated an observe–reason–act–verify framework for autonomous software-supply-chain defense. These studies demonstrate the potential of agentic security workflows, while also highlighting challenges such as reduced performance on complex cases, token overhead, and the need for human intervention.

However, existing approaches do not effectively combine structural, semantic and contextual analysis within a unified vulnerability detection framework. We integrate these approaches through specialized agents and confidence-aware vulnerability inference, while providing explanations and patch recommendations within the GitHub pull-request workflow.

III. PROPOSED SYSTEM AND METHODOLOGY

This section explains the proposed system in three steps for every component: *why* it is needed, *what* it does, and *how* it does it. The goal of the whole system can be stated in one sentence: when CodeSheriff reports that a change is vulnerable with a probability of 87%, changes given that score should turn out to be vulnerable about 87% of the time. A probability with this property is called *calibrated* [21], [22]. Detection accuracy is still important, but calibration is what allows a developer to trust a number instead of ignoring an alert.

A. Design Principles

Four principles guide every design decision in CodeSheriff.

- 1) **Different agents, different failures.** Four agents that fail in the same way add no new information. Each agent is therefore built on a different technique: rules, a language model, repository history, and execution.
- 2) **Agents work blind.** No agent sees the output of another agent. If one agent read another agent’s finding, the two opinions would become correlated, and the fusion step would count the same evidence twice.
- 3) **Every number is measured.** Every likelihood ratio and the alert threshold are fitted on labelled data. There are no hand-set fallback values. If the fitted file is missing, the system stops instead of guessing.
- 4) **Being unable to look is not the same as finding nothing.** An agent that could not run must not lower the probability of a vulnerability. Only an agent that ran and found nothing may do so.

B. System Overview

Fig. 1 shows the complete workflow. When a developer opens or updates a pull request, GitHub sends a webhook to the CodeSheriff API. The API verifies the signature of the request, records an audit, puts the audit identifier in a queue, and replies within three seconds. A separate worker process then takes the audit from the queue, downloads the changed files, and divides the change into one unit per changed function. The four agents analyse every unit independently. The Bayesian inference engine combines their statements into one probability for each possible weakness. If this probability crosses the fitted alert threshold, the patch module tries to draft

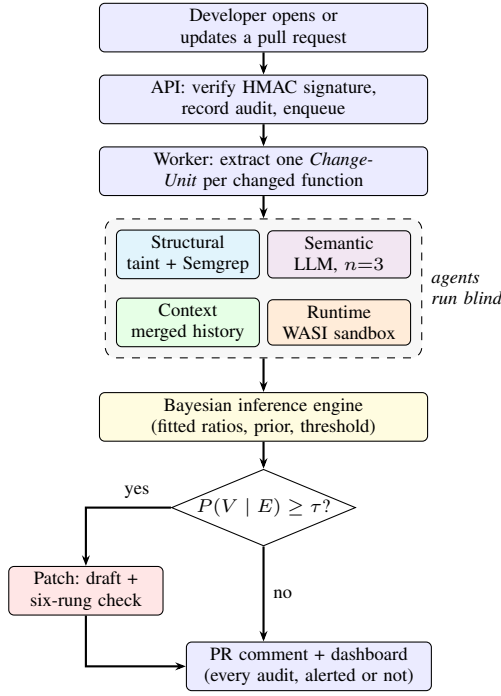


Fig. 1. Workflow of CodeSheriff. The API only verifies and queues; all analysis happens in a separate worker. The four agents never see each other’s output.

and verify a fix. Finally, the result is posted as a comment on the pull request and stored for the dashboard.

C. Unit of Analysis: the Changed Function

Why: A diff fragment is too small to reason about, and a whole file is too large and mixes unrelated code. Also, the fused probability is fitted on labelled functions, so production must analyse the same kind of object.

What: CodeSheriff analyses one *ChangeUnit* per changed function. A *ChangeUnit* holds the file path, the function name, the old and new source of the function, and the list of changed lines.

How: The worker downloads the base and head versions of each changed file. It parses both with *tree-sitter*, which tolerates the syntax errors that are common in unfinished commits, and computes the changed lines by comparing the two versions. Each changed line is mapped to the outermost function that contains it. Changes outside any function form a single module-level unit, because hard-coded credentials usually live at module level. A function that is too large for the analysis budget is not cut short; the agents report that they could not analyse it. Truncating it would produce confident findings from half-read code.

D. The Four Agents

Every agent has exactly the same interface: it receives one *ChangeUnit* and returns a list of statements. Table I summarises what each agent relies on and when it is expected to fail.

TABLE I
THE FOUR AGENTS AND HOW EACH ONE CAN FAIL

Agent	Basis of decision	Fails when
Structural	Rule-based data-flow proof; no LLM	The bug has no syntactic pattern
Semantic	Knowledge of a language model	The model invents a bug or agrees too easily
Context	The repository’s own merged code	The repository has no relevant history
Runtime	Direct observation in a sandbox	The code cannot run in a sandbox

1) *Structural Agent: Why:* Many injection flaws follow a clear pattern: data that the caller controls reaches a dangerous call without being cleaned. Such a flow can be proven mechanically, without any model.

What: The agent reports a weakness when an untrusted value can reach a dangerous operation (a *sink*), such as an SQL query, a shell command, or a file path.

How: It builds a definition–use graph of the function and follows data from each function parameter to known sinks. Every parameter is treated as untrusted, because the function signature is the trust boundary of the unit. Each sink rule states which argument positions are dangerous; for example, only the first argument of `cursor.execute` is dangerous, which is how a parameterised query is correctly treated as safe. A validating guard such as “if the value is not in the allowed list, raise an error” clears the taint, while a simple “is None” check does not. A second backend, *Semgrep* [23], runs pattern rules on the same function. Because both backends read the same text with a similar technique, their results are merged by taking the stronger one, and they count as *one* witness in fusion.

2) *Semantic Agent: Why:* Some weaknesses, such as a missing authorization check, have no fixed syntax. Finding them requires understanding what the function is meant to do.

What: A hosted language model is asked for the intent of the function, its trust boundaries, and the security rule (invariant) that the change breaks.

How: The model is sampled three times with different seeds. The agent’s confidence is the share of samples that report the same weakness, so a finding seen in all three samples is stronger than a finding seen in one. Two safeguards control the known failure modes of language models. First, the analysed code is wrapped between random markers generated for every request, so text inside the code cannot pretend to be an instruction (prompt injection). Second, a *hallucination gate* rejects a finding if its weakness type is out of scope, if it names a different file, if the dangerous call it quotes does not appear word-for-word in the code, or if its line numbers lie outside the function. Worked examples of safe code are included in the prompt so that the model learns that “no vulnerability” is an acceptable answer.

3) *Context Agent: Why:* A change can be dangerous only because of what the repository normally does. For example,

removing an ownership check from one endpoint is a regression if all similar endpoints perform that check.

What: The agent reports a security control that the repository’s merged code establishes and the changed function does not apply.

How: Merged functions are stored as vector embeddings (384 dimensions, computed locally) in PostgreSQL with pgvector. For each unit, similar merged functions of the *same* repository are retrieved. A control is accepted as a rule only if the same function applied it before, or if at least two different sibling functions apply it; one neighbour’s habit is treated as coincidence. The agent reports only authorization weaknesses (CWE-862 and CWE-639). This narrow scope is deliberate: it keeps the agent from repeating what the other agents already say.

4) *Runtime Agent:* *Why:* The strongest evidence that untrusted data reaches a dangerous operation is to see it happen. However, running untrusted pull-request code is exactly how CI systems are attacked, so it must be done in strict isolation.

What: The agent executes the changed function once and reports which dangerous operations an untrusted value actually reached.

How: The function runs inside a WebAssembly [24] sandbox built on Wasmtime [25] and the WebAssembly System Interface (WASI), which gives software fault isolation [26]. The sandbox has no network (a module that asks for a socket fails to load), an empty environment, and no host files. Three limits bound each run: instruction fuel, wall-clock time, and memory. Every parameter receives a unique random token as its value. Because the token is ordinary text, it survives string formatting and concatenation. If the token appears in the argument of a dangerous call, the agent reports a detection. If the run can only reach that call because the harness had to answer a check it could not evaluate, the agent abstains instead of reporting.

E. Three Kinds of Agent Statements

Why: Most tools give a yes/no answer per agent. This hides an important difference: an agent that *looked and found nothing* has provided evidence of safety, while an agent that *could not look* has provided no evidence at all.

What and how: Each agent returns one of three statements for each unit, as shown in Table II. A silence also lists the weakness types the agent is able to check, so that, for example, the structural agent’s silence does not count against an authorization flaw it has no rules for.

F. Bayesian Evidence Fusion

Why: Majority voting treats every agent as equally reliable and ignores how rare vulnerabilities are. Bayes’ rule instead weighs each statement by how reliable that kind of statement has been in the past, and starts from a realistic base rate.

What: For every candidate weakness in a unit, the engine computes the posterior probability $P(V | E)$ that the change is vulnerable (V) given the statements (E) of the four agents.

TABLE II
THE THREE KINDS OF STATEMENT AN AGENT CAN MAKE

Statement	Meaning	Effect on probability
Detection	Agent ran and found a weakness, with a score	Increases it
Silence	Agent ran and found nothing	Decreases it
Abstention	Agent could not run (timeout, no tool, too large)	None (factor = 1)

How: Each agent’s statement is first placed into one of four *cells*: a detection with a score of at least 0.8 (high), between 0.5 and 0.8 (medium), below 0.5 (low), or silence. For each agent a and cell z , a likelihood ratio (LR) states how much more often that cell occurs for vulnerable code than for safe code:

$$\Lambda_a(z) = \frac{P(z | V)}{P(z | \neg V)} \approx \frac{(k_V + 1)/(N_V + m)}{(k_S + 1)/(N_S + m)} \quad (1)$$

Here k_V and k_S count the vulnerable and safe labelled examples that fell into cell z , N_V and N_S are the totals, and m is the number of cells. Adding one to each count (Laplace smoothing) avoids dividing by zero when a cell was never seen. An LR above 1 is evidence for a vulnerability, below 1 is evidence against, and exactly 1 is neutral. Two further rules keep the ratios honest: a cell that was never observed receives exactly 1, and every LR is clamped to $[0.05, 20]$ so no single agent can dominate. An abstention always receives exactly 1.

The prior probability π is turned into odds and multiplied by one LR per agent:

$$O(V | E) = \frac{\pi}{1 - \pi} \prod_{a=1}^4 \Lambda_a(z_a) \quad (2)$$

The odds are then converted back into a probability:

$$P(V | E) = \frac{O(V | E)}{1 + O(V | E)} \quad (3)$$

The product in (2) always has exactly four factors, one per agent, whether an agent spoke or not. Because silences contribute factors below 1, the posterior can go down as well as up.

Worked example. With the declared prior $\pi = 0.03$, the prior odds are 0.031. Suppose the structural agent gives a medium detection ($\Lambda = 15.56$), the semantic agent a high detection ($\Lambda = 9.48$), the context agent abstains ($\Lambda = 1$), and the runtime agent gives a high detection ($\Lambda = 6.92$). The odds become $0.031 \times 15.56 \times 9.48 \times 1 \times 6.92 \approx 31.6$, and the posterior is $31.6/32.6 \approx 0.97$. If instead only the semantic agent had detected and the other three had abstained, the posterior would be 0.227, which is exactly the fitted alert threshold. In plain words, the threshold means: *a strong semantic finding that no other agent contradicts is just enough to alert.*

TABLE III
VERIFICATION LADDER FOR A SUGGESTED PATCH

Rung	Check
1	The patch is valid Python (parsed, never executed)
2	The function keeps the signature its callers depend on
3	The patch uses no name that the file does not define or import
4	The patch actually changes the function
5	The structural and runtime agents that detected the weakness no longer detect it
6	The same agents find no new weakness in the repaired function

G. Patch Suggestion and Verification

Why: An alert is more useful when it comes with a fix, but an unchecked fix produced by a language model can break the code or leave the flaw in place.

What: For every finding above the threshold, the patch module drafts a repaired function, checks it against a fixed ladder of six rungs (Table III), and, if it passes, posts it as a GitHub *suggested change*. CodeSheriff never commits, pushes, or merges.

How: The model receives the same protected prompt as the semantic agent and returns only the repaired function. If a draft fails, the next attempt receives the list of reasons for rejection, written by CodeSheriff, and never the rejected draft itself, since that draft is shaped by untrusted code. At most three drafts are tried. Every rung has three possible outcomes: passed, failed, or not run, so a check that could not run is never shown as passed. A verified fix is published only if all the lines it replaces lie inside the pull request’s changed lines, because GitHub accepts suggestions only there. The repository’s own test suite is never executed, because doing so would need network and filesystem access that the sandbox exists to deny.

IV. IMPLEMENTATION DETAILS

A. Software Architecture

CodeSheriff is written in Python 3.12 as a monorepo of nine packages and three applications. Table IV lists the main technologies. The system runs as two separate processes with separate credentials. The API process holds the GitHub webhook secret and the sign-in secret, and never analyses code. The worker process holds the language-model key and runs all agents. A task passed between them carries only an audit identifier. An import checker in the build enforces that no agent imports another agent, the database, or the fusion engine.

B. Security of the Pipeline

The webhook signature (HMAC-SHA256) is checked on the raw request body before the body is parsed. Source code is kept only in memory; the database stores findings, statements, and hashes of the source, never the source itself. No file path chosen by the pull-request author is written into the pull-request comment. The dashboard only reads data, and every

TABLE IV
IMPLEMENTATION TECHNOLOGY STACK

Layer	Technology
API and queue	FastAPI, Celery, Redis
Storage	PostgreSQL 16, pgvector, SQLAlchemy, Alembic
Parsing	tree-sitter (extraction), Python <code>ast</code> (patch check)
Structural	Custom taint engine (networkx), Semgrep
Semantic	Hosted Gemini Flash-Lite model, $n = 3$ samples
Context	Sentence-Transformers <code>bge-small-en-v1.5</code>
Runtime	Wasmtime + WASI, CPython 3.12 compiled to WebAssembly
Dashboard	Next.js, TypeScript, Tailwind, Recharts
Quality gates	pytest (1249 tests), mypy strict, ruff, import-linter

query is limited to the GitHub installations of the signed-in user. The alert threshold is shown on the dashboard but cannot be changed there, because a per-repository threshold would silently change the meaning of past alerts.

C. Labelled Dataset

Why: Calibration can only be measured against ground truth. *What:* A hand-written corpus of 76 Python functions arranged as 38 *twin pairs*. Each pair holds the same function twice, once vulnerable and once fixed, so a false positive on the safe twin is directly visible. The corpus covers the ten in-scope weakness classes of the Common Weakness Enumeration (CWE) [27]: path traversal (22), OS command injection (78), cross-site scripting (79), SQL injection (89), code injection (94), unsafe deserialization (502), insecure direct object reference (639), hard-coded credentials (798), missing authorization (862), and server-side request forgery (918). *How:* Pairs are assigned to three splits by pair, so the two twins never fall into different splits (Table V). The protocol is strict: likelihood ratios are fitted only on the *calibration* split, the threshold is selected only on the *validation* split, and the *test* split is evaluated exactly once, at the end. No agent can import the corpus, so no agent can read a label.

D. Declared Base Rate

Because every vulnerable case has a safe twin, the corpus is half vulnerable by construction. This does not describe real pull requests, in which vulnerable changes are rare. The prior is therefore *declared* as $\pi = 0.03$ and is not measured from the corpus. The likelihood ratios in (1) are conditioned on the label, so they do not depend on the share of vulnerable cases; this is why a balanced corpus is the right shape for fitting them. All reported metrics are weighted so that vulnerable claims account for 3% of the weight, which estimates behaviour in production rather than on the balanced corpus.

TABLE V
CORPUS COMPOSITION (UNITS PER SPLIT)

CWE	Pairs	Calibration	Validation	Test
CWE-22	3	4	0	2
CWE-78	3	2	2	2
CWE-79	3	4	0	2
CWE-89	3	6	0	0
CWE-94	3	4	2	0
CWE-502	3	4	2	0
CWE-639	6	6	2	4
CWE-798	3	4	0	2
CWE-862	8	8	4	4
CWE-918	3	4	2	0
Total	38	46	14	16

E. Experimental Environment

Experiments were run on a machine with an Intel Core i5 processor and 16 GB RAM. The fitting and evaluation use recorded language-model responses, so a rerun makes no network call and reproduces every number exactly. Every fitted artifact stores a hash of the corpus and of the split assignment it was fitted on, so a later edit to the data shows up as a stale fit.

V. RESULT ANALYSIS AND DISCUSSION

A. Evaluation Metrics

Detection quality is reported as precision (P), recall (R), and F_1 . Calibration is reported with two standard measures. The *Brier score* [28] is the mean squared difference between the predicted probability p_i and the true label $y_i \in \{0, 1\}$:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2 \quad (4)$$

The *Expected Calibration Error* (ECE) [21] groups predictions into ten equal-width bins B_b and compares the average confidence with the observed frequency of vulnerabilities in each bin:

$$\text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{N} |\text{freq}(B_b) - \text{conf}(B_b)| \quad (5)$$

For both measures, lower is better, and 0 is perfect. A statement about one weakness in one unit is called a *claim*. The calibration split yields 49 claims and the validation split yields 15 claims (7 vulnerable).

B. Individual Agent Performance

Before fusion, each agent was measured alone on the calibration split, only on the cases it is designed to detect, and on all safe twins for false positives. Table VI shows that every agent reached full recall on its own scope without a single false positive. The semantic agent is measured differently, because a language model can fail in ways that a rule cannot: none of its accepted findings named a sink that was absent from the code, no injected instruction changed its answer, and it correctly left 94% of safe twins without a finding.

TABLE VI
STAND-ALONE AGENT MEASUREMENT (CALIBRATION SPLIT)

Agent	Recall on own scope	Safe twins	False positives
Structural (taint)	13/13	23	0
Context	5/5	11	0
Runtime	9/9	23	0
Semantic: 0 hallucinated sinks, 0% injection success, 94% of safe twins correctly passed			

TABLE VII
FITTED LIKELIHOOD RATIOS PER AGENT AND CELL

Agent	High	Medium	Low	Silence
Structural	2.22	15.56	–	0.065
Semantic	9.48	0.56	0.74	0.304
Context	2.00	4.00	2.00	0.167
Runtime	6.92	–	–	0.115

C. Fitted Likelihood Ratios

Table VII lists the likelihood ratios fitted on the calibration split using (1). The table can be read directly: a medium structural detection makes a vulnerability 15.6 times more likely, while a structural silence makes it about 15 times *less* likely (0.065). Two cells behave in an interesting way. A low-scoring semantic detection (0.74) and a medium-scoring one (0.56) are evidence of *safety*. This means that when the model reports a weakness in only one or two of its three samples, the code was more often safe than vulnerable. A voting system would count such a finding as a “yes” vote; the fitted ratio correctly treats it as weak evidence against. Cells marked “–” were never observed and contribute exactly 1.

D. Comparison with Baselines

CodeSheriff is compared with two baselines that use the *same* agent statements, so the only difference is how the statements are combined. *Majority vote* alerts according to the share of agents that detected, ignoring abstentions. *Prior only* ignores all agents and always predicts the 3% base rate. All systems use the same threshold of 0.227. Table VIII reports the results on both splits, and Fig. 2 shows F_1 and Brier score on the validation split.

On the validation split, CodeSheriff reaches $F_1 = 0.923$ with perfect precision, alerting on 6 of 15 claims. Majority vote over exactly the same statements reaches only $F_1 = 0.142$: it finds every vulnerability, but most of its alerts are false, because a single agent’s detection is enough to trigger it and there is no prior to hold it back. It is also five times worse calibrated (ECE 0.141 versus 0.027). The prior-only baseline shows why ECE must not be read alone: it scores a perfect ECE of 0 while detecting nothing. The Brier score exposes the difference, since CodeSheriff’s 0.013 is less than half of the prior’s 0.029. CodeSheriff is therefore both *calibrated* and *useful*. The same ordering holds on the larger calibration split.

TABLE VIII
CODESHERIFF VERSUS BASELINES USING IDENTICAL AGENT
STATEMENTS. BEST PER COLUMN IN BOLD.

Split	System	ECE↓	Brier↓	P↑	R↑	F_1 ↑
Valid.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.141	0.059	0.076	1.000	0.142
	CodeSheriff	0.027	0.013	1.000	0.857	0.923
Calib.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.092	0.068	0.161	0.957	0.276
	CodeSheriff	0.009	0.010	1.000	0.696	0.821

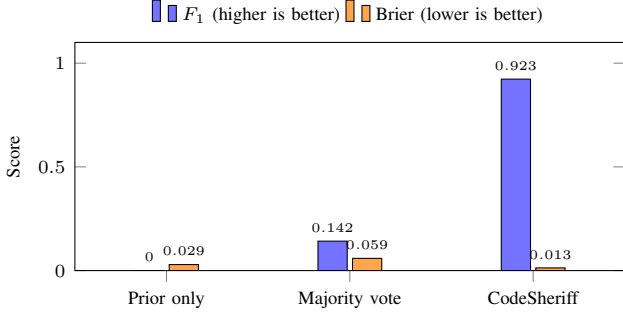


Fig. 2. F_1 and Brier score on the validation split.

E. Reliability Diagram

Fig. 3 plots the average predicted probability in each bin against the observed share of vulnerable claims on the validation split. A perfectly calibrated system lies on the diagonal. Most claims receive a very low probability and are indeed safe (bottom-left point, 8 claims). High-probability claims are all vulnerable (top-right points). In the middle range, two bins lie above the diagonal, which means the system was slightly *under*-confident there: claims given 23% and 34% were in fact vulnerable. One claim given 13% was safe. These middle bins contain only one or two claims each, so they carry very little weight in the ECE; a larger dataset is needed to judge this region reliably.

F. Selecting the Alert Threshold

The threshold was chosen on the validation split by sweeping every distinct posterior value and keeping the one with the best weighted F_1 (Fig. 4). Below 0.227, recall is complete but one safe claim with a posterior of 0.13 is alerted, and because safe code is far more common in production, that single false alarm lowers precision to 0.17. Above 0.227, precision stays perfect but true vulnerabilities start to be missed. The value 0.227 is the point where both are balanced. Because the threshold was chosen on this split, these validation figures are selection-time estimates and not a held-out measurement.

G. Ablation Study

To see what each agent contributes, one agent at a time is removed. A removed agent is treated as an abstention, so it contributes a factor of exactly 1. Table IX and Fig. 5 show the results on both splits.

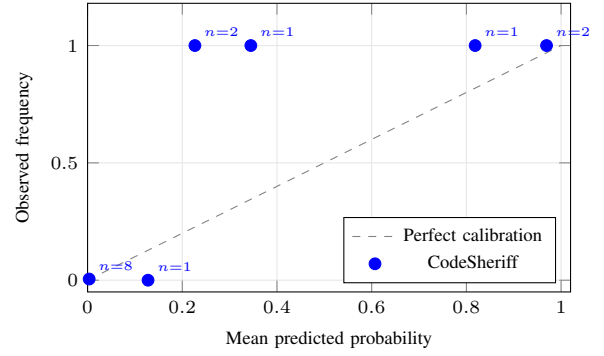


Fig. 3. Reliability diagram on the validation split (15 claims).

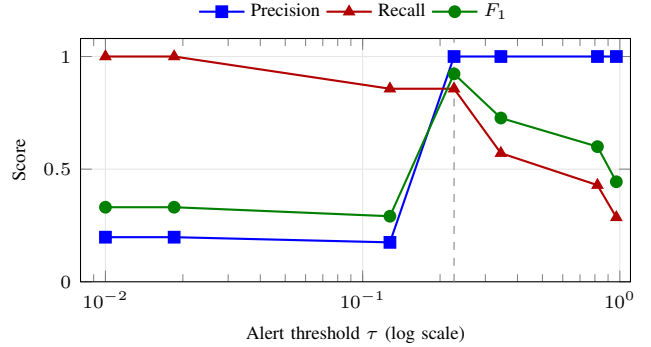


Fig. 4. Precision, recall and F_1 against the alert threshold on the validation split. The dashed line marks the selected $\tau = 0.227$.

The two splits disagree about which agent matters most, and this disagreement is itself the main finding. On the validation split, removing the semantic agent collapses F_1 from 0.923 to 0.157. On the calibration split, removing the structural agent causes the largest drop, from 0.821 to 0.473, because precision falls to 0.359. No single agent is sufficient on both splits, which supports the use of heterogeneous agents. Removing the context or the runtime agent does not change the final decisions on either split. This is reported as a null result and not as proof that these agents are redundant: both agents have narrow scopes (the context agent covers two CWEs), so their evidence usually confirms a decision that another agent has already made. They still move the posterior; for example, removing the runtime agent raises the calibration-split ECE from 0.009 to 0.011.

H. Qualitative Comparison with Related Work

A direct numerical comparison with published systems is not meaningful, because they are evaluated on different languages, weakness sets and metrics. Table X therefore compares capabilities. To our knowledge, CodeSheriff is the first of these systems to combine heterogeneous agents with a calibrated probability, sandboxed execution, and verified patch suggestions inside the pull-request workflow.

TABLE IX
LEAVE-ONE-AGENT-OUT ABLATION AT $\tau = 0.227$

Configuration	Validation			Calibration		
	P	R	F_1	P	R	F_1
Full model	1.000	0.857	0.923	1.000	0.696	0.821
– Structural	1.000	0.714	0.833	0.359	0.696	0.473
– Semantic	0.096	0.429	0.157	1.000	0.609	0.757
– Context	1.000	0.857	0.923	1.000	0.696	0.821
– Runtime	1.000	0.857	0.923	1.000	0.696	0.821

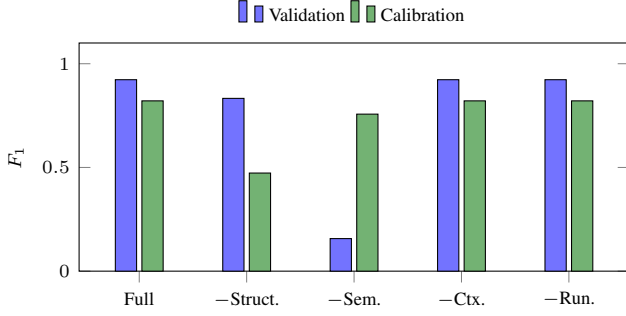


Fig. 5. F_1 after removing one agent at a time.

I. Limitations

The results should be read with the following limits in mind. First, the corpus is small, written by the authors, Python-only, and limited to ten CWEs, so the figures describe this corpus and not arbitrary pull requests. The validation split has only 15 claims, which is too few to attach a useful confidence interval. Second, the held-out test split has not yet been evaluated; it is reserved for a single final run. Third, Semgrep has no build for the Windows machine used for fitting, so the structural ratios describe the taint engine alone; a Linux image with all four agents active has been prepared for the final run. Fourth, the 3% base rate is declared, not measured, and every weighted figure scales with it. Finally, external tools such as Semgrep, Bandit and a single-pass language model have not yet been scored under the same protocol.

VI. CONCLUSION AND FUTURE SCOPE

This paper presented CodeSheriff, a security auditor for GitHub pull requests in which four agents, based on four different techniques, analyse each changed function independently, and a Bayesian engine combines their statements into a calibrated probability. The design separates an agent that found nothing from an agent that could not look, fits every likelihood ratio and the threshold on labelled data, and never lets one agent see another’s output. On the validation split, CodeSheriff achieved an F_1 of 0.923 with perfect precision and an expected calibration error of 0.027, while majority voting over the identical agent statements reached only 0.142 and was five times worse calibrated. The ablation study showed that no single agent is sufficient on both data splits. Alerts are

TABLE X
CAPABILITY COMPARISON WITH RELATED SYSTEMS (✓: REPORTED; –: NOT REPORTED IN THE CITED WORK)

Capability	[5]	[14]	[18]	[19]	Ours
Multiple agents	–	–	✓	✓	✓
Distinct technique per agent	–	–	–	–	✓
Calibrated confidence	✓	–	–	–	✓
Repository context	–	✓	–	–	✓
Sandboxed execution	–	–	–	–	✓
Verified patches	–	–	–	–	✓
PR integration	–	–	–	–	✓

accompanied by patch suggestions that pass a fixed six-step verification ladder and are never committed automatically.

Several directions remain for future work:

- Evaluate the held-out test split exactly once, on the Linux image in which all four agents, including Semgrep, are active, after re-fitting the structural ratios there.
- Score Semgrep, Bandit and a single-pass language model under the same weighted protocol, to complete the external comparison.
- Enlarge the corpus with real vulnerability-fixing commits, so that confidence intervals and a sensitivity analysis over the base rate and threshold become meaningful.
- Extend support beyond Python to JavaScript and TypeScript, which the tree-sitter based extraction already allows.
- Use the same sandbox to release CI/CD secrets only to verified code, and measure the acceptance rate of suggested patches by developers.

REFERENCES

- [1] G. Li and Y. Yang, “On the code vulnerability detection based on deep learning: A comparative study,” *IEEE Access*, 2024.
- [2] S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, “Code-smash: Source-code vulnerability detection using siamese and multi-level neural architecture,” *IEEE Access*, 2024.
- [3] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, “Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis,” *IEEE Access*, 2024.
- [4] B. K. E. Abbasi and A. Shameli-Sendi, “Leveraging key source code metrics for predicting software vulnerabilities with machine learning,” *IEEE Access*, 2026.
- [5] R. A. Almutairi and N. Abbadeni, “Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code,” *IEEE Access*, 2026.
- [6] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, “Finetuning large language models for vulnerability detection,” *IEEE Access*, 2025.
- [7] K. Tamberg and H. Bahsi, “Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study,” *IEEE Access*, 2025.
- [8] D. Gniaciak and T. Szandala, “Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection,” *IEEE Access*, 2025.
- [9] B. Gülmez, “Code generation with large language models: a survey from neural program synthesis to autonomous software development,” *Applied Intelligence*, vol. 56, no. 6, p. 200, 2026.

- [10] Y. Chen, Y. Shen, T. Wang, S. Ou, R. Wang, Y. Li, and Z. Pan, "Software defect detection using large language models: a literature review," *Frontiers of Computer Science*, vol. 20, no. 6, p. 2006202, 2026.
- [11] H. Su, Z. Xu, Y. Zhang, and Q. Tan, "Source code vulnerability detection based on deep learning: a review," *Cybersecurity*, vol. 9, no. 1, p. 2, 2026.
- [12] L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, "A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models," *IEEE Access*, 2025.
- [13] G. Antal, L. Értekes, N. Szolnoki, and P. Hegedűs, "Evaluating retrieval-augmented generation for llm-based vulnerability detection: An empirical study on real-world java vulnerabilities," *IEEE Access*, 2026.
- [14] P.-C. Huang, C.-H. Mao, and C.-M. Lai, "Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification," *IEEE Access*, 2026.
- [15] H. Gao, D. Zhang, and L. Chen, "Scout: Llm-driven vulnerability detection via interactive repository exploration," *IEEE Access*, 2026.
- [16] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, "Enhanced code reviews using pull request based change impact analysis," *Empirical Software Engineering*, vol. 30, no. 3, p. 64, 2025.
- [17] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, "Advanced smart contract vulnerability detection via llm-powered multi-agent systems," *IEEE Transactions on Software Engineering*, 2025.
- [18] Y. Li, K. Joshi, X. Wang, and E. Wong, "Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement," in *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2025.
- [19] K. P. Reddy, J. S. Dhatteval *et al.*, "Agentic artificial intelligence with knowledge-based reasoning for automated vulnerability triage in collaborative software repositories," in *2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies (IC2PCT)*. IEEE, 2026, pp. 493–498.
- [20] T. A. Syed, M. Riyaz Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, "Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation," in *2025 International Conference on Computer and Applications (ICCA)*, 2025.
- [21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [22] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.
- [23] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," <https://semgrep.dev>, accessed: Sep. 2026.
- [24] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [25] Bytecode Alliance, "Wasmtime: A fast and secure runtime for WebAssembly," <https://wasmtime.dev>, accessed: Sep. 2026.
- [26] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient software-based fault isolation," in *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*, 1993, pp. 203–216.
- [27] MITRE Corporation, "Common weakness enumeration (CWE)," <https://cwe.mitre.org>, accessed: Sep. 2026.
- [28] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.